(4). Logical operators (Comparison operators) :-

=> the purpose of logical operator is that "to combine two OR more Relational Expressions".

-> if two or more Relational Expressions are connected with logical operator then it is called "Logical Expression".

-> The Result of logical Expression is either True or false

-> The Logical Expression is called "compound Test Condition" and whose result can be either True or False.

-> In python programming, we have 3 types of logical operators They are:

1. and
2. or
3. not.

① 'and' operator:-

syntax :- $\underbrace{\text{varname} = (\text{RelExpr1}) \text{ and } (\text{RelExpr2})}$

=> The functionality of 'and' operator is shown in the following truth table.

RelExpr1	RelExpr2	RelExpr1 and RelExpr2
True	False	False
False	True	False
False	False	False
True	True	True

Example ①

```
>>> True and False --- False
>>> False and True --- False
```
same.

Example ②

```
>>> 10 > 20 and 2>1 ---- False (short circuit evaluation).
>>> 2>3 and 4>2 and 3>2 --- False (short circuit evaluation)
>>> 2>1 and 4>6 and 5>3 and 2>4 --- False (short circuit evaluation)
>>> 2>1 and 3>2 and -3>-4 ---- True (full length evaluation)
>>> True and 34>56 and 3>2 and 3>7 ---- False (short circuit evaluation)
```

*short circuit Evaluation in the case of 'and' operator

→ if two or more Relational Expressions connected with 'and' operator (called logical expression). and if the initial relational expression is false then PVM will not evaluate Rest of Relational Expression and the Result of entire logical expression.

is considered as False. This process of Evaluation is called short circuit Evaluation.

② 'or' operator

syntax : Varname = (Real Expr1) or (Real Expr2)

→ The functionality of 'or' operator is shown in the following Truth Table.

Real Expr1	Real Expr2	Real Expr1 or Real Expr2
True	False	True
False	True	True
True	True	True
False	False	False.

Example

>>> 10>2 or 30>20 or 40>60 - - - - True (short circuit)

>>> 10>20 or 4>3 or 5>6 or 6>4 - - - True (short circuit)

>>> 10>20 or 40>50 or 20>30 - - - False (full length)

>>> True or 3!3 or 4!=5 or 6!=6 - - - True (short circuit)

>>> True or False or True or False - - - True (short circuit)

③ 'not' operator

syntax 1: not Real Expr.

syntax 2: not ((Real Expr1) or (Real Expr2))

syntax ③: not ((Real Expr1) and (Real Expr2))

Real Expr1	not Real Expr1
True	False
False	True

Examples

```
>>> not True - - - false
>>> not False - - - True.
>>> not (10>2 and 3>2 and 4>2) - - - false
>>> not (10>2 or 3>2 or 4>2) - - - false
>>> not (10>20 and 3>20 and 4>2) - - - True
>>> not (10<2 or 3<2 or 4<2) - - - True.

>>> not 123 - - - - false
>>> not "KVR" - - - - false
>>> not "2-2" - - - - false
>>> not "10-30" - - - - False
>>> not 10-10 - - - - True
>>> not "not" - - - - False
>>> not "" - - - - True
>>> not " " - - - - false
```

Special points

```
>>> 100 and -100 - - - -100.
>>> 100 and 0 and 200 - - - 0
>>> 0 and 200 and -200 - - - 0
>>> "KVR" and "PYTHON" and "Django" - - - 'DJANGO'
>>> "false" and "false" and "True" and "False" - - - "false".
>>> "False and "false" and "True" and "PYTHON" - - - 'PYTHON'
= >>> 100 or 200 - - - - 100.
>>> 100 or 0 or -100 - - - 100.
>>> false or True or 10>20 - - - True
>>> "python" or "java" or "DSC" - - - 'python'
= >>> 100 or 200 and 300 or -100 - - - 100
>>> 100 and 200 or 300 and -100 - - - 200.
>>> 100 or 300 and 200 and 0 or "python" - - - 100
>>> not 100 and 300 or 100 or False and False - - - 100.
```

4.logical Operators:-

In python programming ,we have 3 types of logical operators they are:-

- a)and
- b)or
- c)not

a)and

- syntax:- varname=(RealExpr1)and(RealExpr2)

In [5]:
```
True and False
```
Out[5]: False

In [6]:
```
False and True
```
Out[6]: False

In [7]:
```
False and False
```
Out[7]: False

In [8]:
```
True and True
```
Out[8]: True

In [9]:
```
10>20 and 2>1
```
Out[9]: False

In [10]:
```
2>3 and 4>2 and 3>2
```
Out[10]: False

In [11]:
```
2>1 and 4>6 and -3>-4
```
Out[11]: False

In [12]:
```
True and 34>56 and 3>2 and 3>7
```
Out[12]: False

b)or

- syntax:- varname=(RealExpr1)or(RealExpr2)

In [13]:
```
10>2 or 30>20 or 40>60
```
Out[13]: True

In [14]:
```
10>20 or 4>3 or 5>6 or 6>4
```

Out[14]: True

In [15]:
```
10>20 or 40>50 or 20>30
```

Out[15]: False

In [17]:
```
True or 3!=3 or 4!=5 or 6!=6
```

Out[17]: True

In [18]:
```
True or False or True or False
```

Out[18]: True

c)not

- synatx 1:- notRealExpr1
- synatx 2:- not((RealExpr1)or(RealExpr2))
- syntax 3:- not((RealExpr1)and(RealExpr2))

In [19]:
```
not True
```

Out[19]: False

In [20]:
```
not False
```

Out[20]: True

In [22]:
```
not(10>2 and 3>2 and 4>2)
```

Out[22]: False

In [23]:
```
not(10>2 or 3>3 or 4>2)
```

Out[23]: False

In [24]:
```
not(10>20 and 3>20 and 4>2)
```

Out[24]: True

In [25]:
```
not(10<2 or 3<2 or 4<2)
```

Out[25]: True

In [26]:
```
not 123
```

Out[26]: False

In [27]:
```
not "kvr"
```

Out[27]: False

In [28]:
```
not "2-2"
```
Out[28]: False

In [29]:
```
not "10-30"
```
Out[29]: False

In [30]:
```
not 10-10
```
Out[30]: True

In [31]:
```
not "not"
```
Out[31]: False

In [32]:
```
not " "
```
Out[32]: False

In [33]:
```
not ""
```
Out[33]: True

In [34]:
```
#special points
100 and -100
```
Out[34]: -100

In [35]:
```
100 and 0 and 200
```
Out[35]: 0

In [36]:
```
0 and 200 and -200
```
Out[36]: 0

In [37]:
```
"kvr" and "pyhton" and "django"
```
Out[37]: 'django'

In [38]:
```
"False" and "False" and "True" and "False"
```
Out[38]: 'False'

In [39]:
```
"False" and "False" and "True" and "python"
```
Out[39]: 'python'

In [40]:
```
100 or 200
```
Out[40]: 100

In [41]:
```
100 or 0 or -100
```
Out[41]: 100

In [42]:
```
False or True or 10>20
```
Out[42]: True

In [43]:
```
"pyhton" or "java" or "dsc"
```
Out[43]: 'pyhton'

In [44]:
```
100 or 200 and 300 or -100
```
Out[44]: 100

In [45]:
```
100 and 200 or 300 and -100
```
Out[45]: 200

In [46]:
```
100 or 300 and 200 and 0 or "python"
```
Out[46]: 100

In [47]:
```
not 100 and 300 or 100 or False and False
```
Out[47]: 100

In [ ]: